CS7.503: Mobile Robotics

Lifelong Mapping & 3D Change Detection

Ch Shreyash 2024121008
Farzana S 2024121006
Prajas Wadekar 2024121009

Abstract

Homes, offices and labs change between visits: chairs are moved, boxes appear and objects are put away. A robot that returns with a map from an earlier visit has to find its pose in that map with no initial guess, find out which objects were moved, added or removed, and update the map accordingly. We propose a training-free lifelong mapping pipeline for this. The map is a point cloud of the room, split into one segment per object and organised as a 3D scene graph, in which each node is an object and edges record spatial relationships. On each visit, the robot builds another scene graph from what it currently sees and matches it to the previous graph. The distance between two objects that did not move is the same in both visits, so the largest group of matched objects whose distances all agree is the set of objects that stayed in place. This gives us the robot's pose. Smaller groups whose distances agree among themselves are objects that moved together. An object without a match is labelled as removed or added only if depth images show that its space is empty; otherwise it is kept and marked as not observed. The stored map is then edited per object, so objects keep their identity and the map size depends on the number of objects rather than the number of visits. We evaluate relocalization, change detection and map size on 3RScan.

1 Introduction // Motivation

Robots in homes, offices and labs return to the same rooms many times, and the rooms change in between: furniture is rearranged and objects are added or put away. When a robot returns, it has to find its pose in the old map with no initial guess, even though some of the landmarks may have moved. It also has to find which objects were moved, added or removed, and tell these apart from objects that are only hidden or outside the area it observed this time. Then, it has to update the map with those changes. Replacing the old map with the new deletes everything that cannot be currently seen, loses which object is which, and moves the map's coordinate frame. Appending every new scan keeps outdated copies and makes the map grow with every visit.

3D scene graphs suit this problem because they store a scene as separate objects with positions and relations, so changes can be described per object [6, 9]. Existing work covers only parts of the problem. Scene-graph mapping systems assume that objects stay in place between runs: Kimera [6] models only agents (people and robots) as moving, and the map update of ConceptGraphs [2] deletes objects that are not re-detected regardless of whether they were in view. Scene-graph alignment finds the pose but not the changes in the map. SGAligner [7] matches the objects of two scene graphs and computes the pose from the matched objects, but it ignores the objects it cannot match, so it does not report what changed. Change detection methods, on the other hand, assume that the two scans are already aligned. Objects Can Move [1] uses the alignment supplied with 3RScan and outputs unlabelled point clusters, without a pre-defined notion of what an object is supposed to look like. DSG [3] uses ground-truth camera poses, has no "moved" state (a moved object becomes a missing object plus a new one), and needs GPU rendering and a large language model. 3D VSG [4] learns how likely each object is to change, but does not check whether it actually did. Finally, none of these methods is evaluated over more than two visits, and none reports how the map size grows over multiple observations.

We propose a pipeline that performs all three tasks on each visit, without training and without a given alignment. It outputs the robot's pose in the old map, the objects that were moved (with their motion), added, removed or not observed, and the updated map.

2 Related Work

Kimera [6] builds a layered scene graph (mesh, objects, places, rooms) where objects are static and only agents move. 3DSSG [9] provides scene graphs for every scan in 3RScan, with one node per object instance and labelled relations between objects, and predicts such graphs from point clouds. ConceptGraphs [2] builds a graph whose nodes are object point clouds with image features, connected by relations from a language model. These methods describe only one scan.

3RScan [8] introduced the task of finding the pose of a given, already segmented object in a later scan of the same room. SGAligner [7] aligns two scene graphs: a network trained on 3RScan embeds each object from its point cloud, relations and attributes, objects with similar embeddings are matched, and the rigid transform between the scans is estimated from point matches on the matched objects. It also aligns a partial scan to a prior map that contains changes, but reports only how well objects are matched, not what changed. When many candidate matches are wrong, PCM [5] and TEASER [10] keep only the largest group of matches in which every pair agrees. PCM applies this to loop closures between two maps, whereas TEASER applies it to point matches, where two matches agree if the distance between their points is the same in both point clouds.

Objects Can Move [1] detects moved, added and removed objects between two aligned scans without change labels. It compares depth maps of the two scans to find candidate changes, fits a rigid motion to each group of points that moved together, and spreads each change to all parts of the scene that share that motion. DSG [3] stores each object as a set of 3D Gaussians and compares renderings with images to label objects stable, appeared or missing. 3D VSG [4] trains a graph neural network on pairs of 3RScan scans to predict how likely an object is to move, change state, or appear or disappear, but does not verify these predictions against the actual changes.

3 Proposed Contributions

C1. We propose a pipeline driven by **object-level maps that are iteratively edited** across visits. By organizing segmented point clouds into a scene graph, our approach retains unobserved regions and preserves individual object identities and change histories.

C2. We introduce a **pairwise distance test** for both relocalization and change detection, operating without training data or prior alignment. By evaluating the relative positions of matched objects, the largest geometrically consistent group establishes the robot's pose, while smaller consistent clusters identify objects that moved together.

C3. We ensure map integrity through **depth-based verification** of removals and additions. Objects are classified as removed or added only if new depth images confirm their physical space is currently empty or was explicitly observed as empty previously. Objects merely hidden or outside the current field of view are safely retained and marked as unobserved.

C4. We **evaluate the pipeline** over multiple visits. Using the 3RScan dataset without ground-truth alignment, we process rooms with three or more scans as continuous visits. This allows us to test relocalization accuracy, change detection, and map growth after every successive observation.

4 Methodology

4.1 Representing the map

The map is represented as a point cloud, divided into one segment per object. A 3D scene graph is built on top of these segments. Each object is a node that stores the object's points, its centre, its bounding box, its class, and its change history. To provide spatial context, the graph also includes nodes representing the room's floor and walls. Edges connect objects through spatial relationships. Figure 1 shows an example.

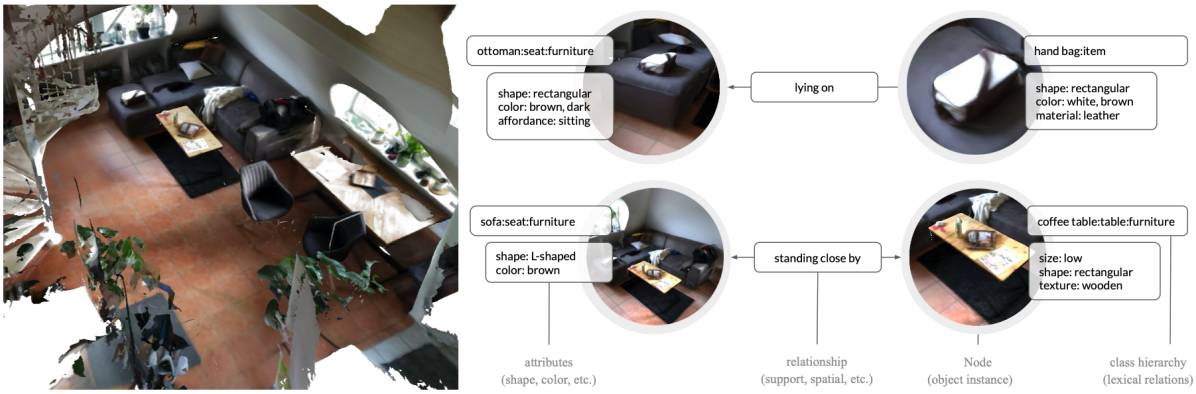


Figure 1: A 3D scene graph of a 3RScan scan, from 3DSSG [9]. Our map has the same structure, but each node also stores the object's points, centre, bounding box and change history.

4.2 Building a graph from a visit

Each visit produces a point cloud of the room in the robot's own coordinate frame. The point cloud is split into object segments, and the properties of each node and the edges are computed from the segments. The graph of the first visit is stored as the map. On later visits, the graph is temporary: it is used in the steps below and then discarded.

4.3 Updating the map on each later visit

Step 1: Candidate matches. Each map object is paired with a few objects of the new visit that look similar in size, shape, height above the floor and class. None of these depend on the robot's pose. Identical objects each get several candidates and Step 2 decides which one is right. Choosing among them by appearance alone may lead to matching the wrong copy, like in SGAligner.

Step 2: Objects that stayed in place. If two objects did not move, the distance between them is the same in both visits, wherever the robot is. For every pair of candidate matches, we check whether the two distances agree, allowing for sensor noise and for the centre of a large object shifting when only part of it is seen. The largest group of matches in which every pair agrees is taken as the set of objects that stayed in place. This search for the largest agreeing group follows PCM and TEASER [5, 10].

Step 3: Robot pose. The rotation and translation that bring the static objects of the new visit onto their positions in the map give the robot's pose. The pose is refined by aligning the points of these objects and of the floor and walls. If too few static objects are found, or the alignment is poor, the method reports that it could not relocalize instead of returning a pose.

Step 4: Objects that moved. Each remaining group whose distances agree is a set of objects that moved together, and its motion is computed as in Step 3. An object that moved alone has no partner to compare distances with, so its old and new point clouds are aligned directly, and it is labelled moved if they fit and Step 5 shows that its old location is now empty.

Step 5: Removed, added, or not observed. A map object with no match was either removed or not observed on this visit. Its points are placed into the new visit's frame using the pose from Step 3 and projected into the visit's depth images. If the camera sees surfaces behind the place where the object was, that space is now empty and the object is labelled removed. If the place is hidden behind something or was never in view, the object is kept and marked not observed. A new object with no match is labelled added if its space was observed empty on an earlier visit. If that space was never observed, it is inserted into the map but not counted as a change.



Figure 2: Two scans of the same room in 3RScan, from [8]: (a) 3D reconstructions, (b) aligned textured meshes, (c) instance segmentation with IDs shared across scans, (d) the alignment of objects that moved, and (e) calibrated RGB-D frames.

Step 6: Editing the map. Objects that stayed in place receive the new points. Moved objects are shifted by their motion. Removed objects are deleted, and added or newly observed objects are inserted. Objects that were not observed are kept unchanged. Each object's change history and the record of empty space are updated and edges are recomputed around edited objects. The map size depends on the number of objects and not the number of visits.

5 Dataset

We use 3RScan [8], which contains 1482 RGB-D scans of 478 indoor rooms, with 2–12 scans per room recorded minutes to months apart. Each scan provides RGB-D frames with camera poses, a textured mesh, instance labels whose IDs stay the same across scans of a room, the transform that aligns it to the room's reference scan, and the rigid motion of each object that moved. We also use 3DSSG [9], which adds a scene graph with object classes and support relations to every 3RScan scan.

6 Evaluation Plan

We evaluate on 3RScan in two settings. In the two-visit setting, the room's reference scan is the map and each rescan is a new visit; the rescan is given a random rotation and shift, so the dataset alignment is used only to score the result. In the multi-visit setting, the scans of every room with three or more scans are processed one after another, and the map is edited after each. For **relocalization**, we report the success rate, which is the share of rescans whose estimated pose is within the standard 3RScan error thresholds [8], and the time needed to compute the pose. For **change detection**, we report precision and recall for each change type (moved, added, removed), counted per object. For **map quality over visits**, we report the size of the map after each visit in the multi-visit setting, and the share of objects in the room that are in the map at the correct place.

7 Project Timeline

- **Phase 1 (Mid Submission - Oct 26):** Load 3RScan and 3DSSG; build graphs from the annotated segments; implement Steps 1–3; report relocalization results in the two-visit setting.
- **Phase 2 (Final Submission - Nov 23):** Implement Steps 4–6; run the multi-visit setting; use predicted segments; write the final report.
- **Phase 3 (Project Demo - Nov 26):** Prepare the presentation and a demonstration of relocalization, change detection and map editing on 3RScan rooms.

References

[1] Aikaterini Adam, Torsten Sattler, Konstantinos Karantzalos, and Tomas Pajdla. Objects can move: 3D change detection by geometric transformation consistency. In *European Conference on Computer Vision (ECCV)*, 2022.

[2] Qiao Gu, Ali Kuwajerwala, Sacha Morin, Krishna Murthy Jatavallabhula, Bipasha Sen, Aditya Agarwal, Corban Rivera, William Paul, Kirsty Ellis, Rama Chellappa, Chuang Gan, Celso Miguel de Melo, Joshua B. Tenenbaum, Antonio Torralba, Florian Shkurti, and Liam Paull. ConceptGraphs: Open-vocabulary 3D scene graphs for perception and planning. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.

[3] Ming Liao, Chao Ye, Jianing Fei, and Weiyang Lin. DSG: Dynamic 3D scene graph construction for embodied agents in changing indoor environments. arXiv preprint arXiv:2609.00619, 2026.

[4] Samuel Looper, Javier Rodriguez-Puigvert, Roland Siegwart, Cesar Cadena, and Lukas Schmid. 3D VSG: Long-term semantic scene change prediction through 3D variable scene graphs. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2023.

[5] Joshua G. Mangelson, Derrick Dominic, Ryan M. Eustice, and Ram Vasudevan. Pairwise consistent measurement set maximization for robust multi-robot map merging. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2018.

[6] Antoni Rosinol, Andrew Violette, Marcus Abate, Nathan Hughes, Yun Chang, Jingnan Shi, Arjun Gupta, and Luca Carlone. Kimera: From SLAM to spatial perception with 3D dynamic scene graphs. *The International Journal of Robotics Research*, 40(12–14):1510–1546, 2021.

[7] Sayan Deb Sarkar, Ondrej Miksik, Marc Pollefeys, Daniel Barath, and Iro Armeni. SGAligner: 3D scene alignment with scene graphs. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.

[8] Johanna Wald, Armen Avetisyan, Nassir Navab, Federico Tombari, and Matthias Nießner. RIO: 3D object instance re-localization in changing indoor environments. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.

[9] Johanna Wald, Helisa Dhamo, Nassir Navab, and Federico Tombari. Learning 3D semantic scene graphs from 3D indoor reconstructions. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.

[10] Heng Yang, Jingnan Shi, and Luca Carlone. TEASER: Fast and certifiable point cloud registration. *IEEE Transactions on Robotics*, 37(2):314–333, 2021.